

# QA og Tests



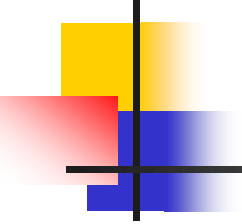
# Software Kvalitet

## Hvad er software kvalitet?

- Diskuter emnet kvalitet
- Forslå en general definition af kvalitet
- Tænk på eksempler for god og dårlig kvalitet ud fra jeres definition af kvalitet



**SOFTWARE QUALITY**

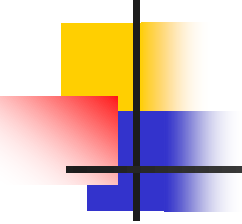


# Quality Definitions

---



- *Quality is meeting or exceeding customer expectations at a cost that represent a value to them – H. James Harrington*
- *Quality should be defined as surpassing customer needs and expectations throughout the life of the product – Howard Gitlow and Shelley Gitlow*
- *Quality is conformance with requirements Philip Crosby*

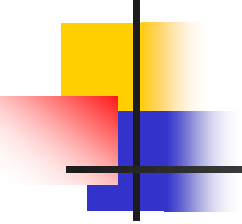


## Øvelse

---

**Beskriv hvordan du vil vurdere kvaliteten af en uddannelsesinstitution.**

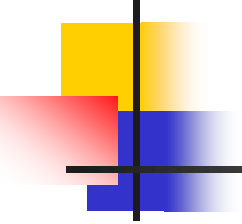
**\*Hvilke faktorer er vigtige?**



# Software Kvalitet

---

- Software Kvalitet kan blive defineret som:
  - "An effective software process applied in a manner that creates a useful product that provides measurable value for those who produce it and those who use it."  
(Pressman)



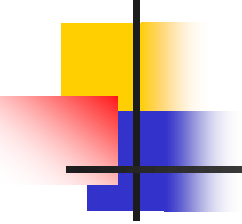
# Effective Software Process

---

An ***effective software process*** establishes the infrastructure that supports any effort at building a high quality software product.

*Consists of:*

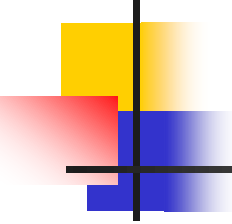
- **Software engineering practices** allow the developer to analyze the problem and design a solid solution—both critical to building high quality software.
- **Project management** help avoid project chaos—a key contributor to poor quality.
- Finally, **umbrella activities** such as reviews have as much to do with quality as any other part of software engineering practice.



# Useful product

---

- A ***useful product*** delivers the **content, functions, and features** that the end-user desires
- But as important, it delivers these assets in a **reliable, error free** way.
- A useful product always satisfies those **requirements** that have been explicitly stated by stakeholders.
- In addition, it satisfies a set of **implicit requirements** (e.g., ease of use) that are expected of all high quality software.



# Adding value for both the producer and user

## HOW?

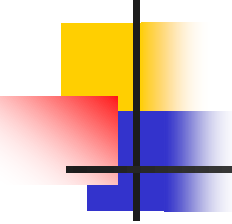
---

- The **software organization** gains added value because high quality software requires
  - **less maintenance effort**
  - **fewer bug fixes**
  - **reduced customer support**
  - **more time for creating new applications and less rework.**
- The **users** gains added value because the application supports business process in a useful way.

The end result is:

- greater software product revenue,
- better profitability when an application supports a business process, and/or
- improved availability of information that is crucial for the business.



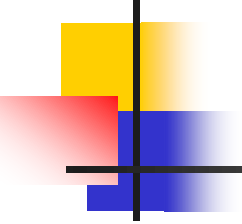


# Spørgsmål?

---

- For at vide om kvaliteten er øget, vil det være nødvendig at kunne måle kvalitet.
  - **Hvordan kan vi måle kvaliteten?**





# Eksempler på kvalitets definitioner

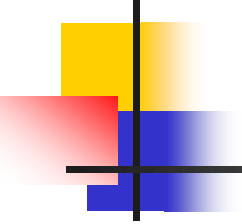
---

## Quality in use model:

- Effectiveness
- Satisfaction
- Freedom from risk
- Context coverage

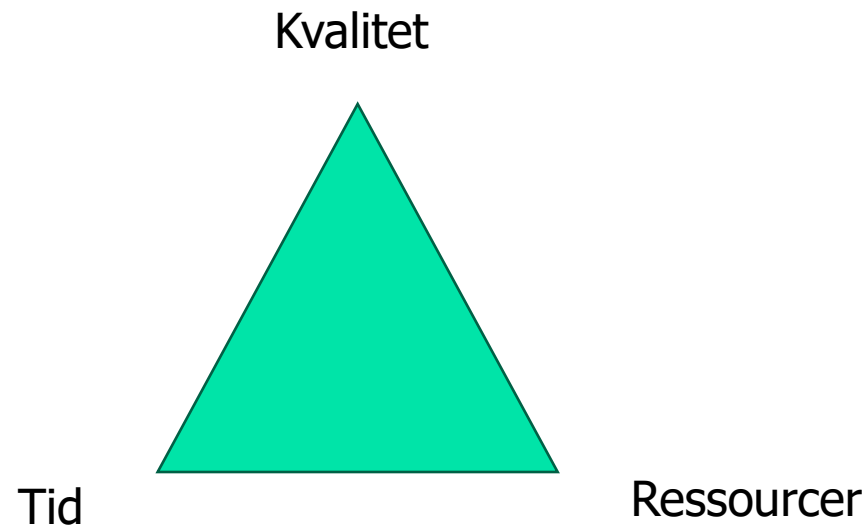
## Product quality:

- Functional suitability
- Performance efficiency
- Compatibility
- Usability
- Reliability
- Security
- Maintainability
- Portability



# Indflydelse på kvalitetsopnåelse

---



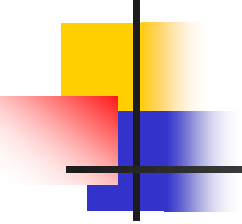
# The Software Quality Dilemma

If you produce a software system that has terrible quality, you lose because no one will want to buy it.



If you spend infinite time, extremely large effort, and huge sums of money to build the absolutely perfect piece of software, then it's going to take so long to complete and it will be so expensive to produce that you'll be out of business anyway.

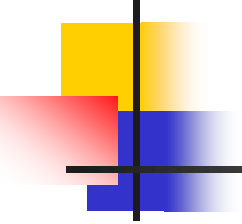
- So people in industry try to get to that magical middle ground where the product is **good enough not to be rejected** right away, such as during evaluation, but also not the object of so much perfectionism and so much work that it would take too long or cost too much to complete.



# “Good Enough” Software

---

- Good enough software delivers **high quality functions and features** that end-users desire, but at the same time it delivers other more **obscure or specialized functions and features that contain known bugs**.
- **Arguments *against* “good enough.” ?**
  - If you deliver a “good enough” (buggy) product, you risk permanent damage to your company’s reputation.
  - You may never get a chance to deliver version 2.0 because bad buzz
  - If you work in certain application domains where software is critical you risk lawsuit



# Cost of Quality

---

## ***Prevention costs*** include

- quality planning
- formal technical reviews
- testing
- training

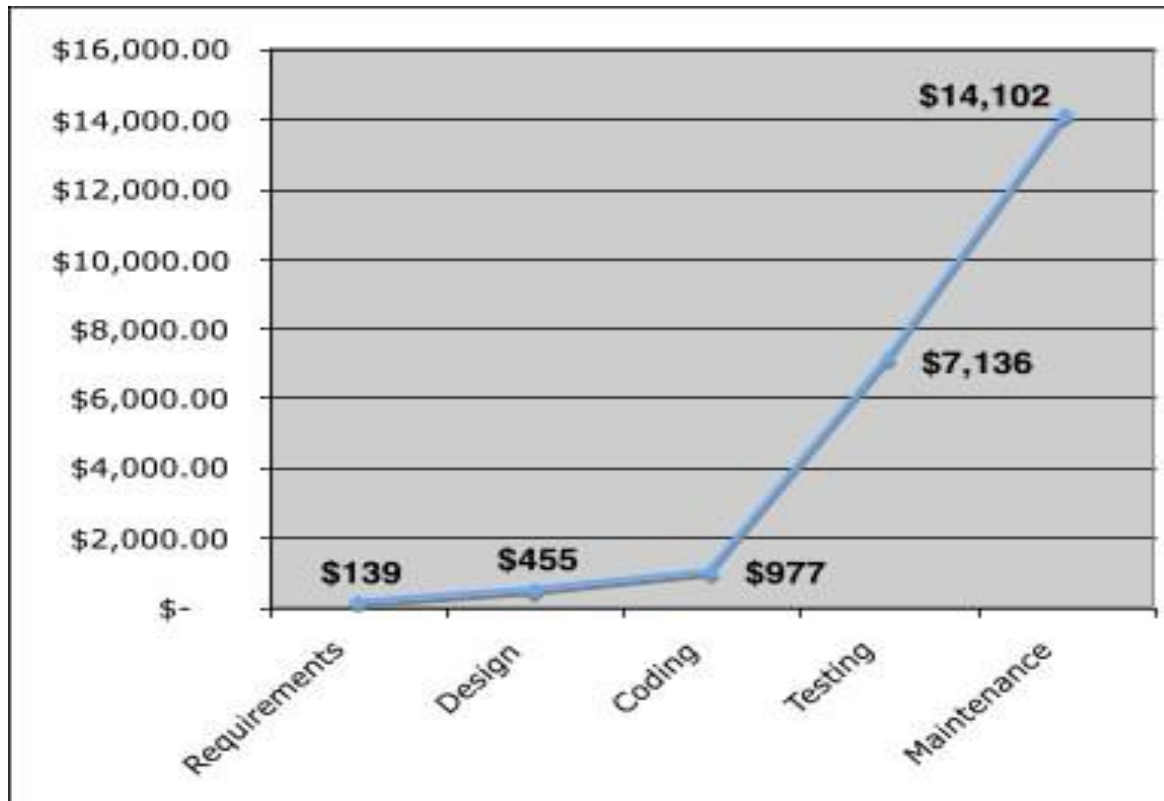
## ***Internal failure costs*** include

- rework
- failure mode analysis
- repair

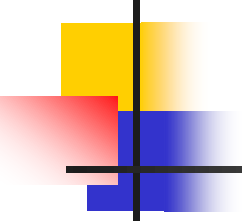
## ***External failure costs*** are

- complaint resolution
- product return and replacement
- help line support
- poor reputation

# Cost



- The relative costs to find and repair an error or defect increase dramatically as we go from prevention to detection to internal failure to external failure costs.



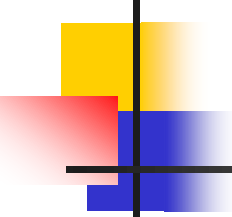
# Achieving Software Quality

---

- **Critical success factors:**

- **Software Engineering Methods**
- **Project Management Techniques**
- **Quality Control**
  - E.g. Reviews, code inspections, test
- **Quality Assurance**
  - systematic monitoring and evaluation of the various aspects of a project to maximize the probability that minimum standards of quality are followed.



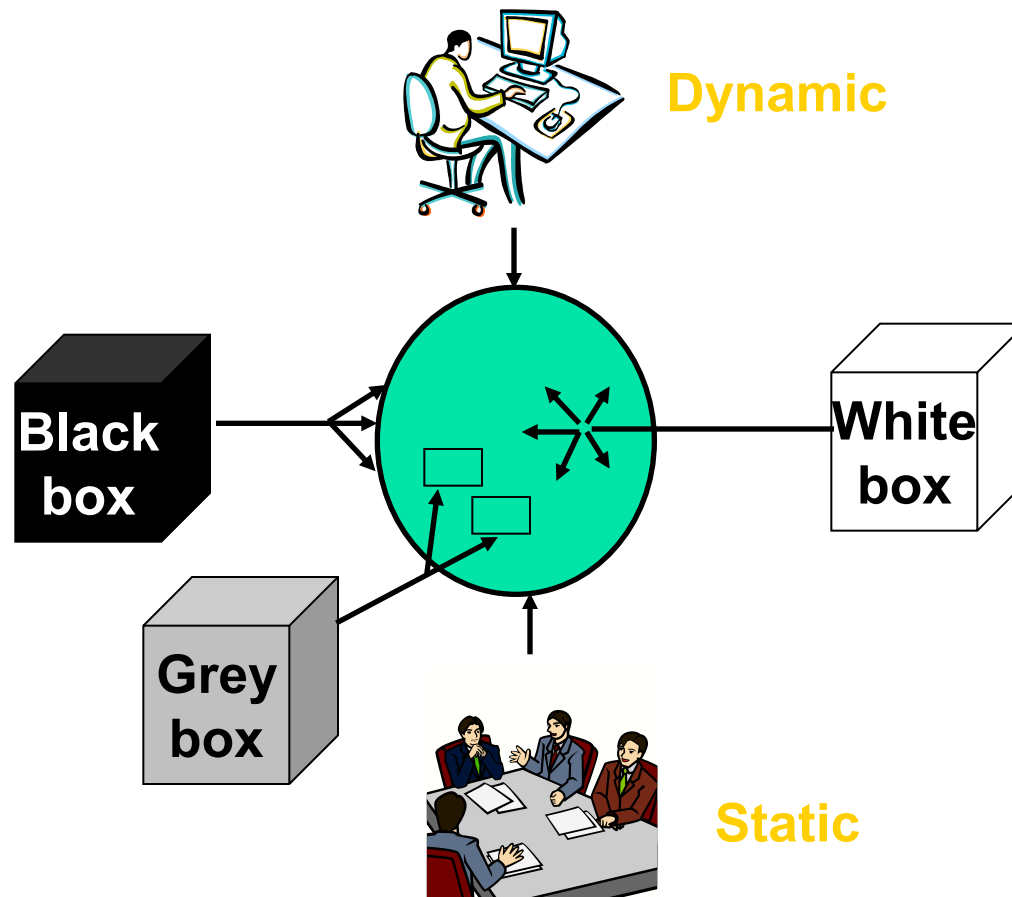


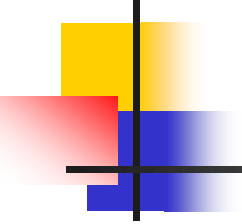
# Hvordan kan I påvirke kvaliteten i udviklingen?

---

- **Aftale kodestandarder**
- **Strukturere koden**
- **Teste koden**
- **Lave review af koden**

# Test techniques

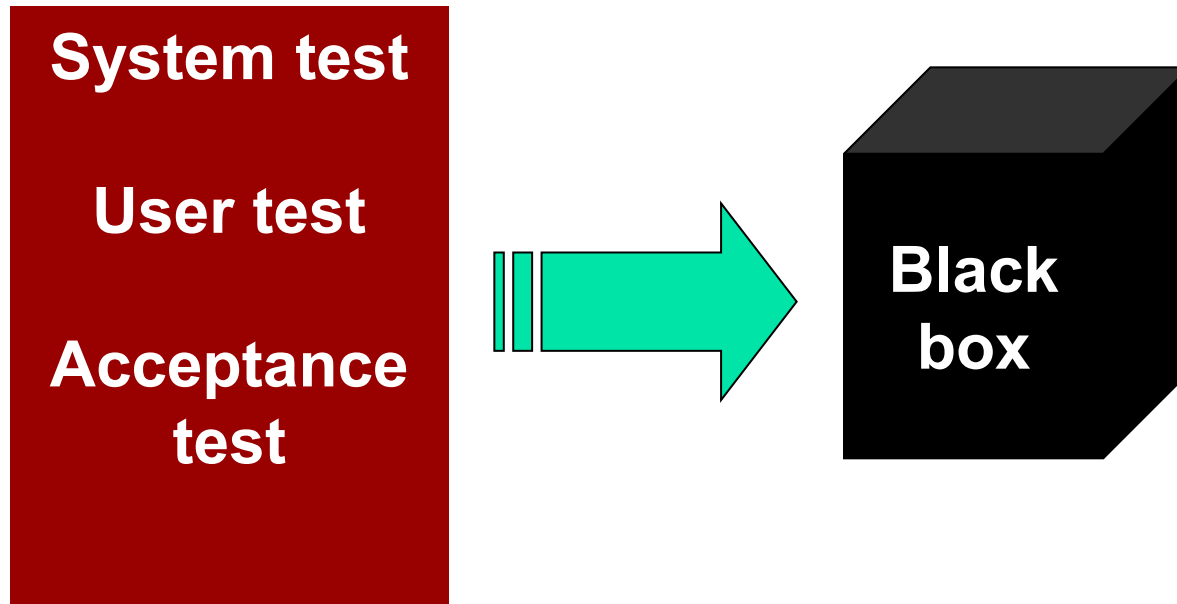




# Blackbox test

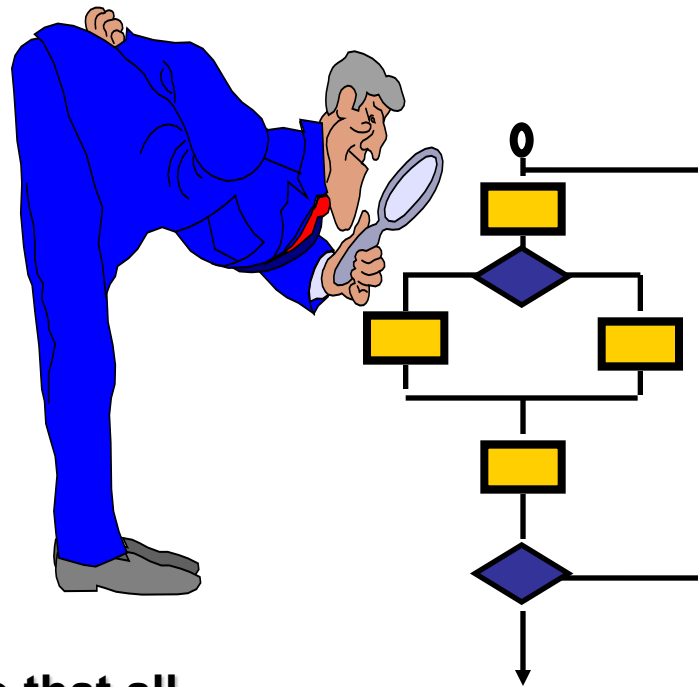
---

**When?**

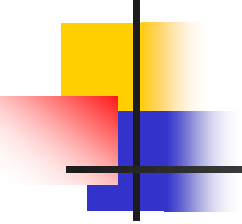


# White-Box Testing

The tester knows the structure of the code

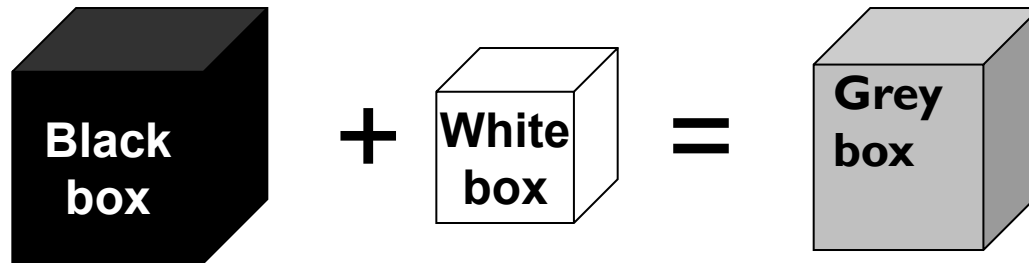


**... our goal is to ensure that all statements and conditions have been executed at least once ...**



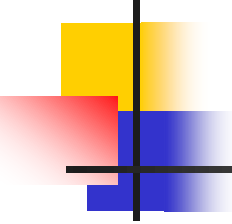
# Greybox test

---



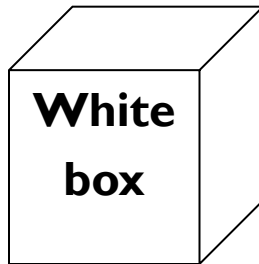
**Example**

Users find risks during their use of the system (Blackbox). Based on the identified risks, chosen sections of the code are inspected (Whitebox)

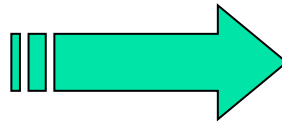
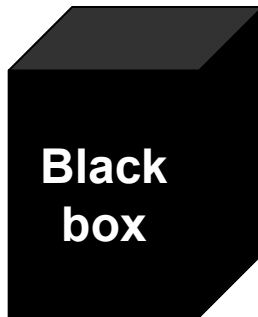


# When are the techniques used?

---



- Unit test
- Integration test



- System test
- Alpha test
- Beta test
- Acceptance test